



Bank4U
Smart banking, simplified.

IT7742 — Assessment 1: Analysis and Prototyping

Sprint 1 Technical Report

Student Name	Keyur
Student ID	20231990
Solution Name	20231990_Keyur_BankSprint1
Course	IT7742
Submission	Assessment 1 — Sprint 1
GitHub Repository	https://github.com/keyur232/bank4u-sprint1

Task 1 — System Architecture

The diagram below shows the full domain model for Sprint 1: two inheritance hierarchies (User roles and Account types), the composition relationship between Customer and Account, and every method override required by the business rules.

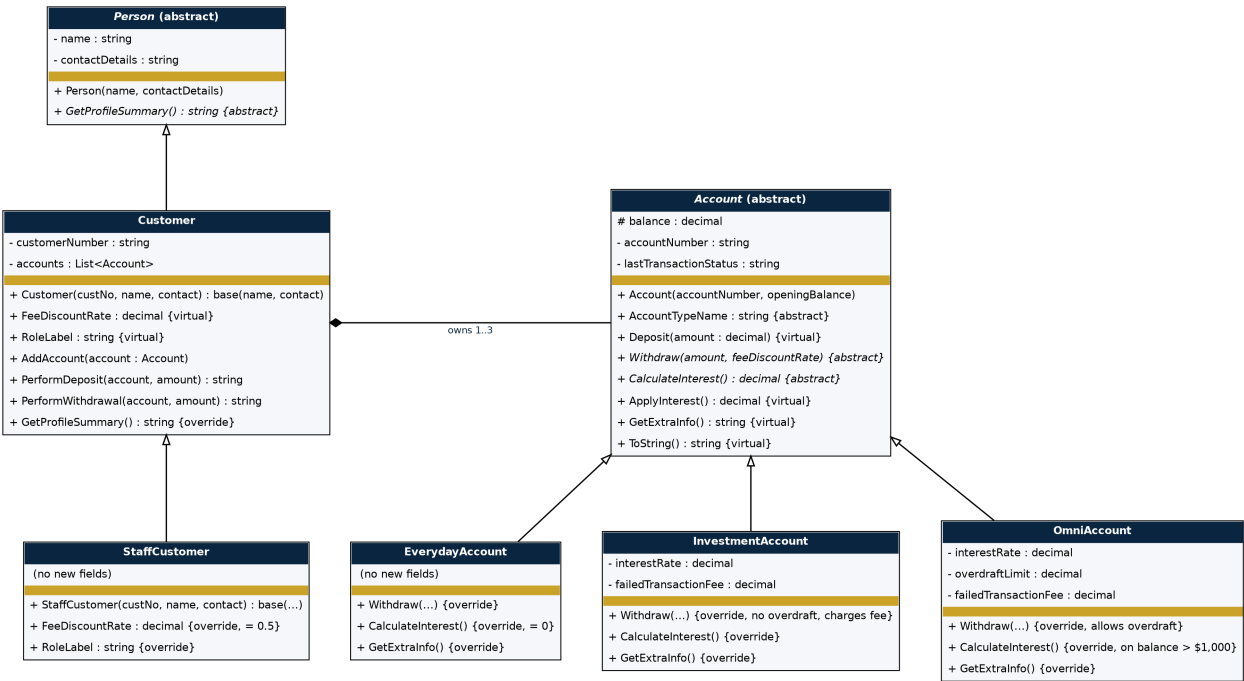


Figure 1 — UML Class Diagram: Bank4U Sprint 1 Domain Model

Technical Justification (295 words)

The domain model uses two parallel inheritance hierarchies, both anchored by an abstract base class, to satisfy the banking scenario's requirement for shared structure with type-specific behaviour. Person and Account are declared abstract because neither is ever meaningful on its own: a Person with no role, or an Account with no interest, fee, or overdraft rule, would be an incomplete and unsafe object to instantiate. Marking them abstract enforces, at compile time, that only a concrete, fully specified type (Customer, StaffCustomer, EverydayAccount, InvestmentAccount, OmniAccount) can ever be created.

The Account hierarchy follows a Template Method style design: the base class defines the overall shape of a transaction (Deposit, Withdraw, ApplyInterest, ToString) while forcing each subclass to supply its own Withdraw and CalculateInterest logic through abstract methods, since these are exactly the rules that differ between account types — no fee versus a fixed fee, no overdraft versus a defined limit, flat interest versus a \$1,000 threshold. Constructor chaining (base(...)) threads the shared accountNumber and openingBalance up to Account, while each subclass constructor adds only its own extra fields such as interestRate, overdraftLimit, or failedTransactionFee.

The User hierarchy (Person to Customer to StaffCustomer) applies the same pattern to the 50% staff discount rule: FeeDiscountRate is a virtual property overridden only in StaffCustomer, so the discount is applied polymorphically wherever a fee is calculated, without any isStaff branching in the GUI or even inside Account. The same technique (RoleLabel, GetExtraInfo) removes every other type check from the Form entirely. Composition (Customer holds a List<Account>) reflects real ownership, since an Account cannot exist independently of its Customer in this model. Encapsulation is enforced throughout via private fields, a protected balance visible only within Account's own hierarchy, and public properties, keeping every calculation inside the domain layer as required.

Task 3 — Professional GUI Prototype: Design Diagrams

A low-fidelity wireframe of the main screen was produced before implementation to plan the visual hierarchy of the header, sidebar, account card, transaction controls, and transaction history — followed by a user-flow diagram mapping the full journey from launch through to a completed transaction.

Figure 2 - Bank4U Low-Fidelity Screen Wireframe (Main Window)

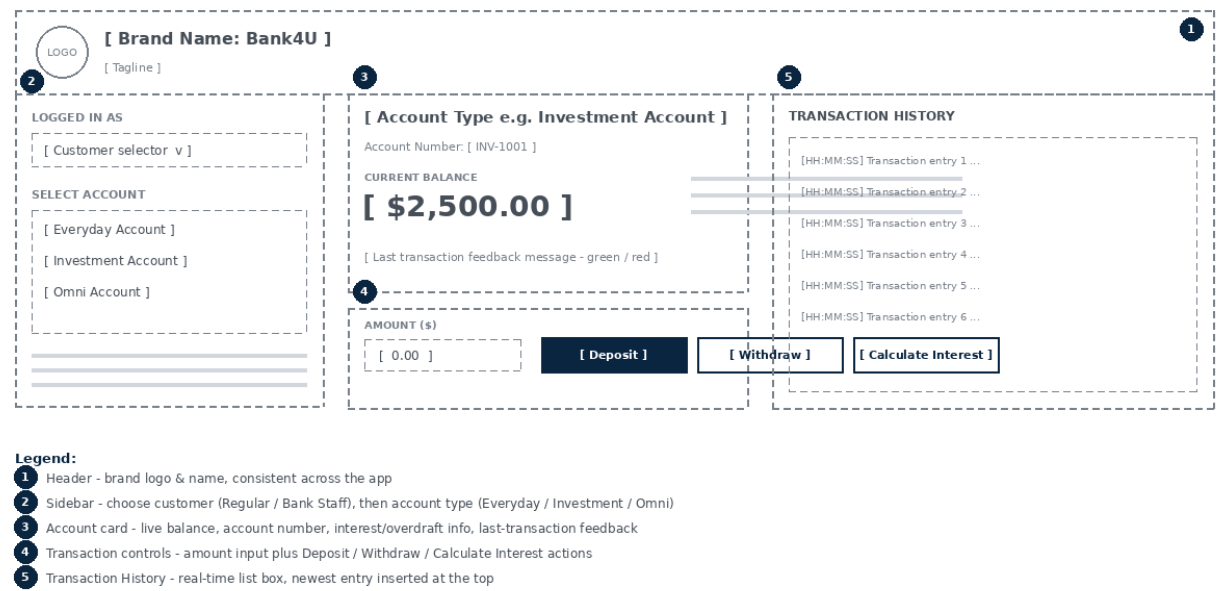


Figure 2 — Low-Fidelity Screen Wireframe (Main Window)

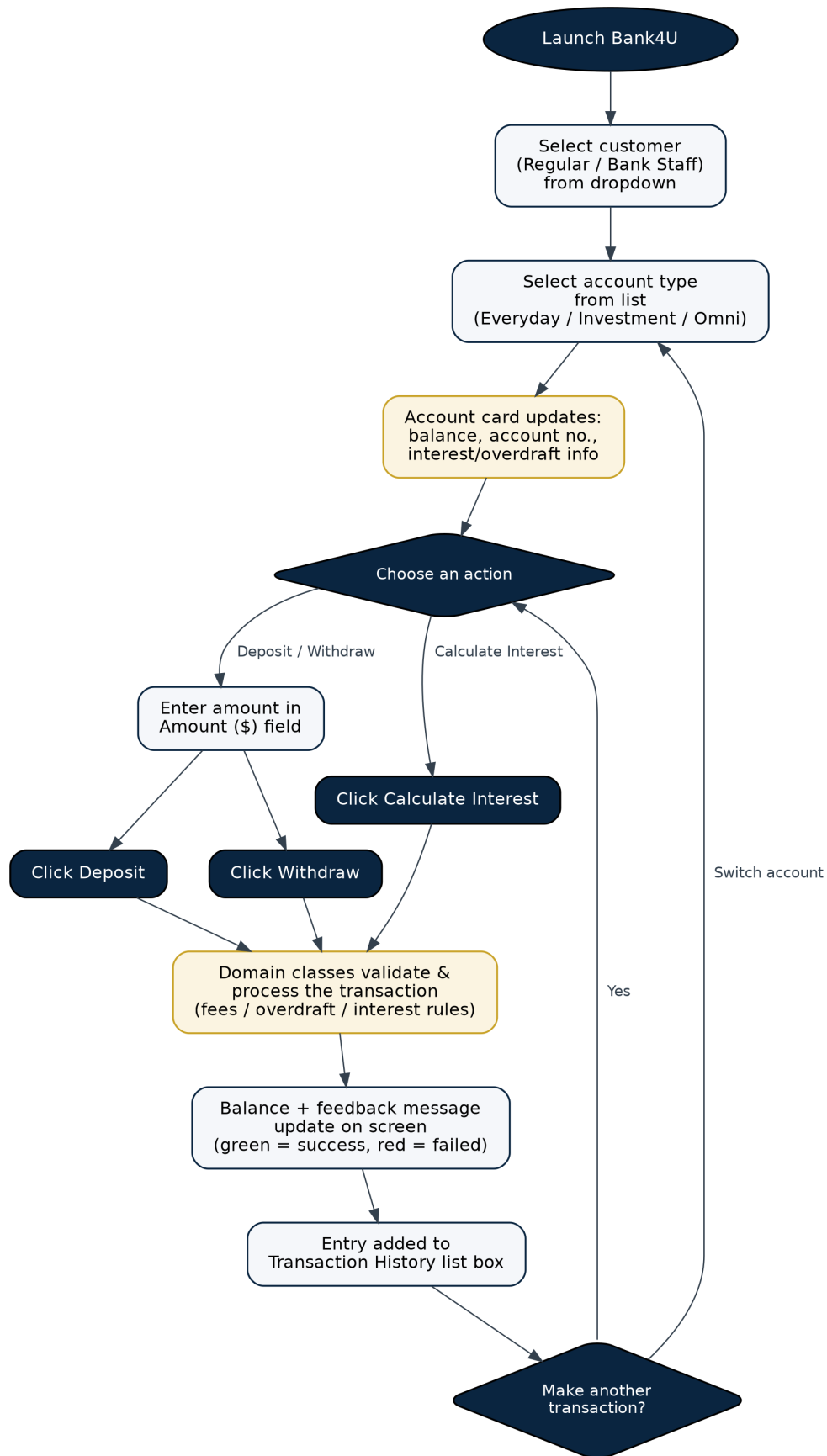


Figure 3 — User Flow Diagram: Account Selection to Transaction Completion

Business Logic Assumptions

- Interest is applied on demand via the "Calculate Interest" action rather than automatically on a timer, since Sprint 1 is a prototype with no persistence or scheduling layer.
- Everyday Account: a withdrawal that exceeds the balance simply fails with no fee, matching the brief's "no transaction fees" rule for this account type.
- Investment Account: no overdraft is permitted; a failed withdrawal (insufficient funds) incurs a fixed fee, discounted 50% for Bank Staff.
- Omni Account: withdrawals are permitted up to balance + overdraft limit; a fee is only charged if the withdrawal exceeds even the overdraft limit.
- Bank Staff are modelled as customers who can also hold accounts (StaffCustomer extends Customer), since the brief does not exclude staff from banking with the same institution — the 50% discount applies to fees on their own transactions.
- Two demo profiles (one Regular Customer, one Bank Staff) are pre-populated purely to demonstrate the discount rule side-by-side; a production system would authenticate an actual logged-in user instead of a selector.
- All monetary values use a single currency and standard decimal rounding; multi-currency support is out of scope for Sprint 1.

Task 4 — Version Control

Private GitHub repository (lecturer/tutor added as a collaborator):

<https://github.com/keyur232/bank4u-sprint1>

The repository contains a staged commit history reflecting progress across the Sprint (initial UML/setup, domain model implementation, GUI/branding, and final polish), rather than a single final upload. See GITHUB_GUIDE.md in the submitted project folder for the exact commands used.

Declaration

The work presented in this assessment is to the best of my knowledge original, except as acknowledged in the text, and the material has not already been submitted, either in whole or in part, for any academic award at this or any other tertiary institution.